ROSA Regional Platform

ROSA-667 - Milestone 2 - Continuous Validation

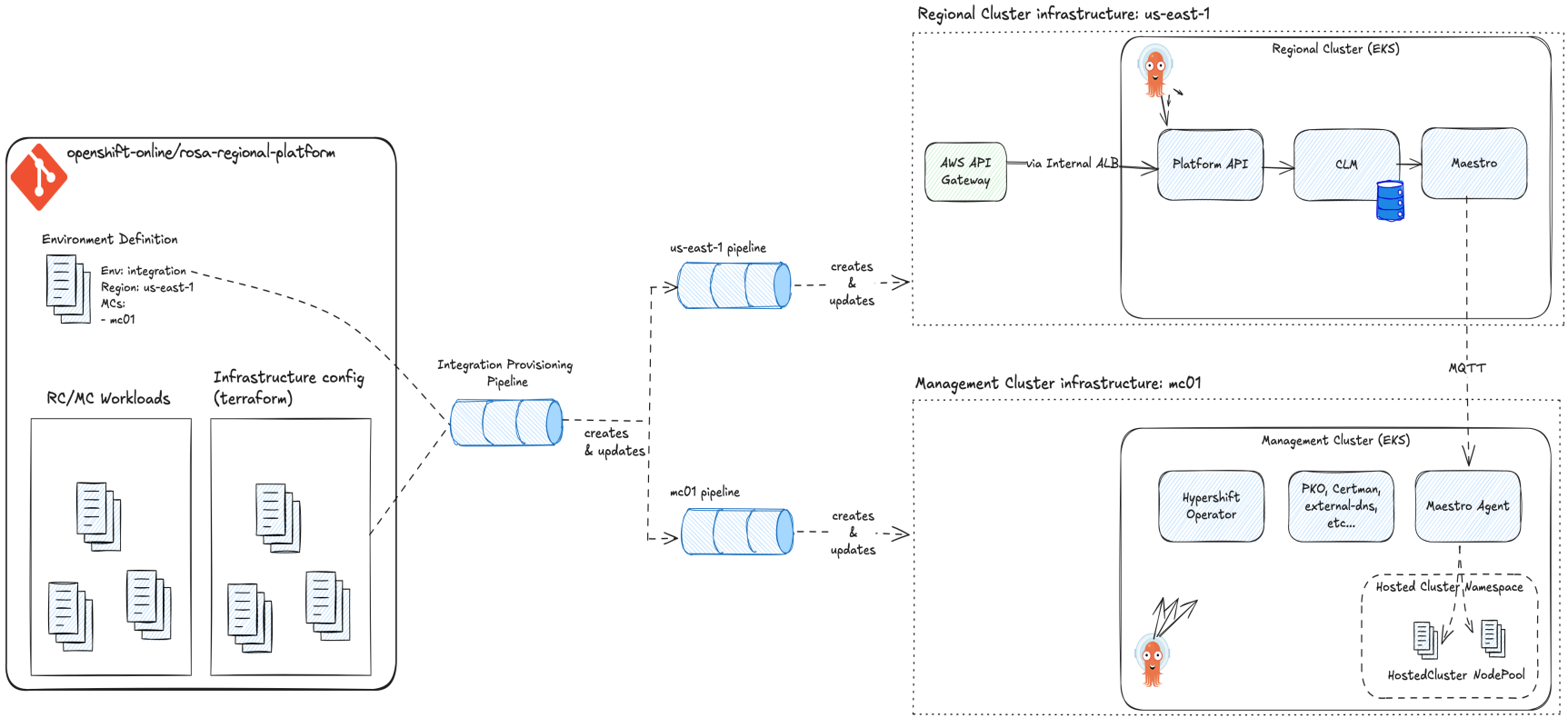
Before building HCP lifecycle or customer-facing APIs, we invested in CI and testing. M2 establishes the testing foundation so that every change going forward is validated automatically.

Evolution from Milestone 1

M1 used **manual Terraform**. Now, everything is **GitOps-driven** via `deploy/`.

- **ArgoCD** syncs workloads to Regional / Management Clusters
- **AWS CodePipelines** handle infrastructure changes and new region/MC provisioning
- Every component (application or infra) that constitutes a region has a **defined version in Git**

Architecture Overview



Integration Environment

Persistent environment in `us-east-1`, consuming from `main` of rosa-regional-platform via **continuous delivery**.

```
awscurl --service execute-api --region us-east-1 https://api.us-east-1.int.rosa.devshift.net/v0/ready

{"status":"ok"}
```

API supports **Management Clusters** and **Manifest Works** CRUD with status feedback. See the OpenAPI spec.

Testing Strategy

We test against two types of environments:

Ephemeral

- Full region (RC + MC + all infra) **provisioned from scratch**, then **torn down**
- Not a simulation - **same CodePipeline** that will provision production

Long-lived

- The **persistent Integration environment**
- Catches issues that only surface over time: **state drift**, **resource leaks**

Pre-merge Validation (every PR)

All jobs run on **OpenShift CI** (Prow + ci-operator).

- `terraform-validate` - validates all Terraform root modules
- `helm-lint` - lints Helm charts
- `check-rendered-files` - verifies rendered deploy files are up to date
- `on-demand-e2e` - provisions an ephemeral environment, runs e2e tests, tears down (manual trigger via `/test on-demand-e2e`)

Nightly Jobs

Job	Runs the Testing Suite against
`nightly-ephemeral`	An ephemeral environment provisioned from `main`
`nightly-integration`	The long-lived Integration environment

Weekly `ephemeral-resources-janitor` cleans up CI AWS accounts via `aws-nuke`.

Results: `nightly-ephemeral` | `nightly-integration`

Testing Suite

Built with **Ginkgo**, reusable against **any environment**.

Today

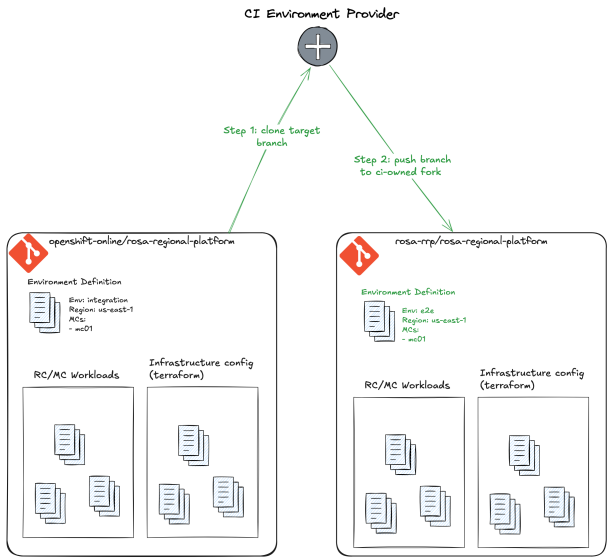
- **Platform API** e2e tests (Manifest Works lifecycle + status feedback)

Next

- **HCP lifecycle**: create, get, update, delete HostedClusters and NodePools
- **Workload validation**: deploy workloads into HCPs and verify they run
- **Customer-facing features**: log forwarding, zero egress, and more

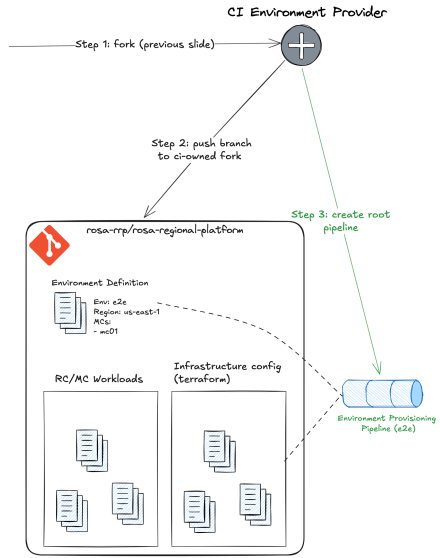
How Ephemeral Tests Work — Fork & Configure

Everything is **GitOps-driven** - CI clones the target branch into a **CI-owned fork** with an `e2e` environment definition.



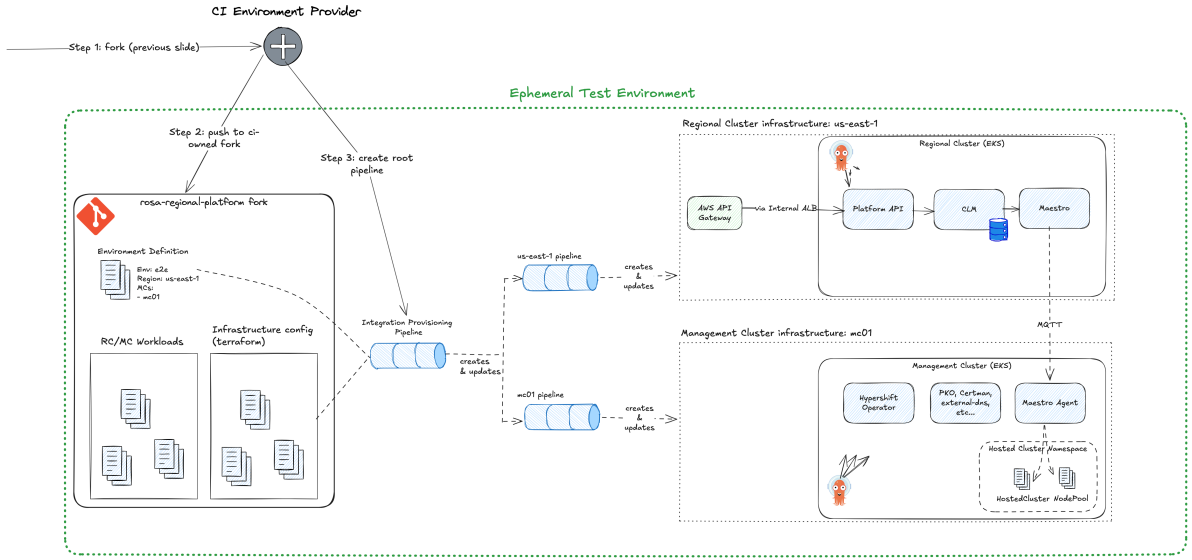
How Ephemeral Tests Work – Create Root Pipeline

A **root pipeline** is created that consumes the CI-owned fork for provisioning.



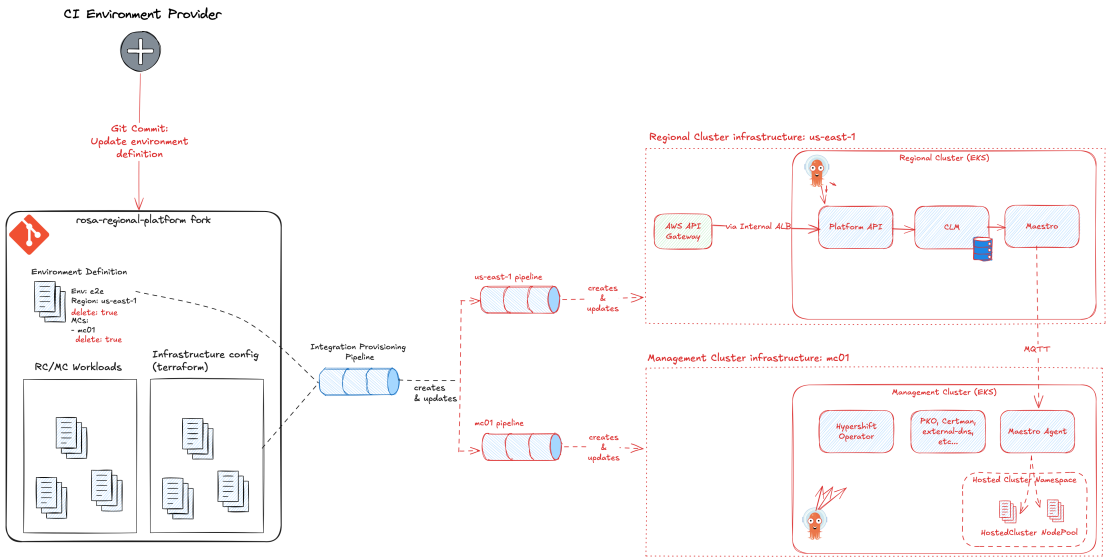
How Ephemeral Tests Work – Provisioned Environment

Environment spun up using **infrastructure and workload config from the original branch**. Testing Suite runs against it.



How Ephemeral Tests Work — Teardown

Teardown is also **GitOps-driven**. `delete: true` is committed to the config, and pipelines tear down all infrastructure.



Pre-merge Tests for Component Repos

Our goal is to hook up ephemeral pre-merge tests to **every component repository** that constitutes the platform.

- Environment provisioned from `main` of rosa-regional-platform
- **Only your component is replaced** with the PR version
- Full Testing Suite runs against it

Milestone 2 Complete - What's Next?

- **Sippy integration** (TRT-2572) - test results dashboard for nightly jobs
- **Pre-merge tests** using ephemeral environments for more repos
- **Open up** the Integration environment for broader internal access

Next milestones:

- ROSA-668 - HCPs Run on EKS MCs (M3)
- ROSA-669 - Observability and Alerting (M4, RHOBS v2)